

IS105 - ENTERPRISE ARCHITECTURE

MR. NIL CARLO M. DELA
CRUZ INSTRUCTOR



CLASS DIAGRAM



AT THE END OF THIS LESSON, YOU SHOULD BE ABLE TO:

- Familiarize with the UML Class diagram.
- Be conscious how important UML Class Diagram is for visualizing and documenting software systems.

WHAT IS A CLASS DIAGRAM IN UML?



INTRODUCTION

- The Unified Modeling Language (UML) can help you model systems in various ways.
- One of the more popular types in UML is the class diagram.
- Popular among software engineers to document software architecture, class diagrams are a type of structure diagram because they describe what must be present in the system being modeled.

CLASS DIAGRAM

- UML was set up as a standardized model to describe an object-oriented programming approach.
- Since classes are the building block of objects, class diagrams are the building blocks of UML.
- The various components in a class diagram can represent the classes that will actually be programmed, the main objects, or the interactions between classes and objects.

BENEFITS OF CLASS

DIAGRAMS Illustrate data models for information systems, no matter how simple or complex.

- Better understand the general overview of the schematics of an application.
- Visually express any specific needs of a system and disseminate that information throughout the business.

BENEFITS OF CLASS

DIAGRAMS

Create detailed charts that highlight any specific code needed to be programmed and implemented to the described structure.

- Provide an implementation-independent description of types used in a system that are later passed between its components.

CLASS DIAGRAM COMPONENTS



CLASS

- A template for creating objects and implementing behavior in a system.
- In UML, a class represents an object or a set of objects that share a common structure and behavior.
- They're represented by a rectangle that includes rows of the class name, its attributes, and its operations.

CLASSES

- When you draw a class in a class diagram, you're only required to fill out the top row—the others are optional if you'd like to provide more detail.

- **Name:** The first row in a class shape.
- **Attributes:** The second row in a class shape.

Each attribute of the class is displayed on a

CLASSES

- **Upper section:** Contains the name of the class. This section is always required, whether you are talking about the classifier or an object.
- **Middle section:** Contains the attributes of the class. Use this section to describe the qualities of the class. This is only required when describing a specific instance of a

CLASSES

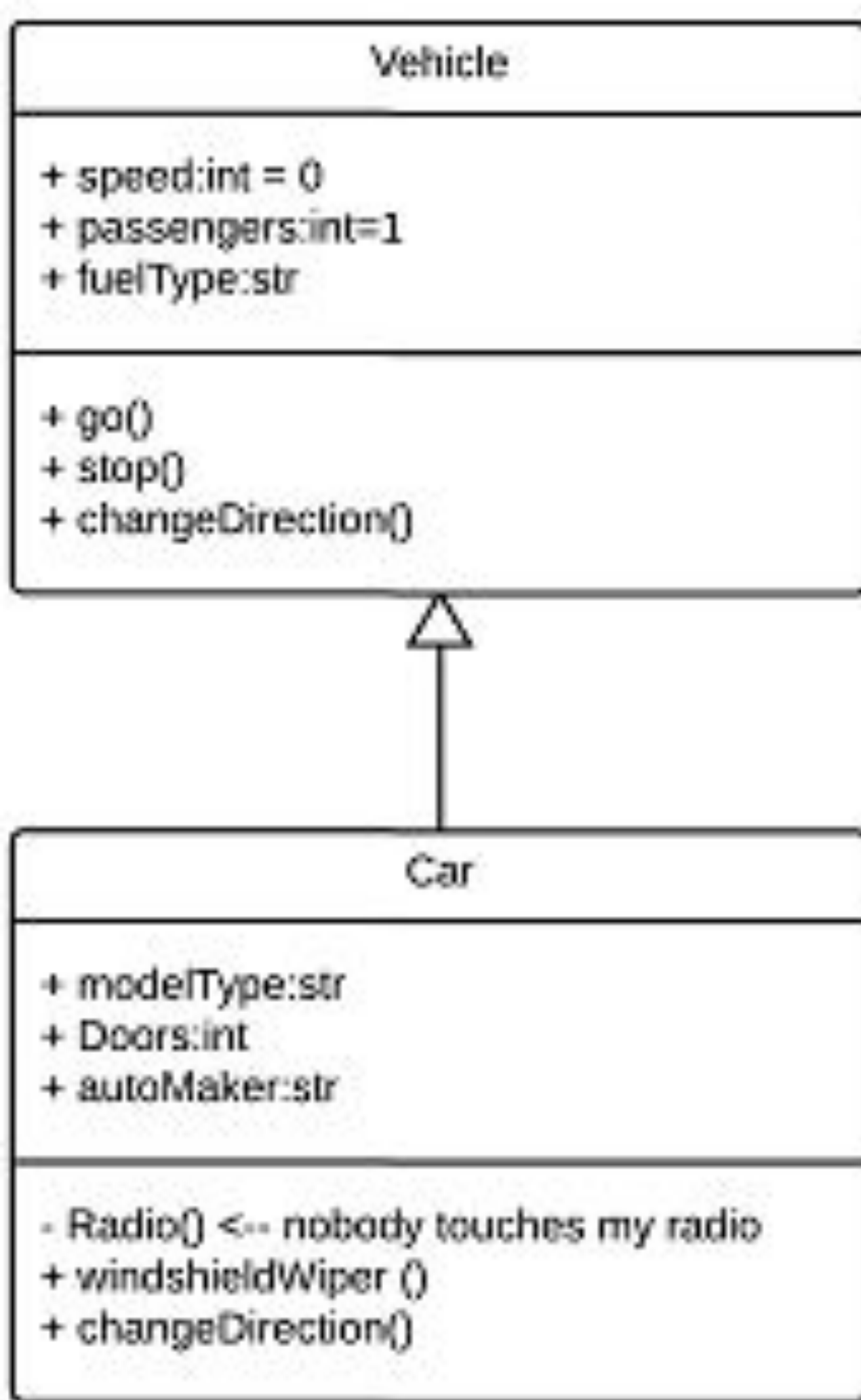
- **Bottom section:** Includes class operations (methods). Displayed in list format, each operation takes up its own line. The operations describe how a class interacts with data.

MEMBER ACCESS MODIFIERS

- All classes have different access levels depending on the access modifier (visibility). Here are the access levels with their corresponding symbols:
- Public (+)
- Private (-)
- Protected (#)
- Package (~)
- Derived (/)
- Static (underlined)

INTERACTIONS

- The term "interactions" refers to the various relationships and links that can exist in class and object diagrams.
- Some of the most common interactions include:
- **Inheritance:** The process of a child or sub-class taking on the functionality of a parent or superclass, also known as generalization. It's symbolized with a straight connected line with a closed arrowhead pointing towards the superclass



In this example, the object "Car" would inherit all of the attributes (speed, numbers of passengers, fuel) and methods (go(), stop(), changeDirection()) of the parent class ("Vehicle") in addition to the specific attributes (model type, number of doors, auto maker) and methods of its own class (Radio(), windshieldWiper(), ac/heat()). Inheritance is shown in a

INTERACTIONS

- **Bidirectional association:** The default relationship between two classes.
- Both classes are aware of each other and their relationship with the other.
This association is represented by a straight line between two classes.

Car

+ modelType:str
+ Doors:int
+ autoMaker:str

- Radio() <-- nobody touches my radio
+ windshieldWiper ()
+ changeDirection()

RoadTrip

+ snacks:str
+ fuelLevel:int
- radioStation:str

+ route(start,end)
- detour()
- pottyBreak()

assignedCar

0..*

0..1

getRoadTrip

INTERACTIONS

- In the example above, the Car class and RoadTrip class are interrelated.
- At one end of the line, the Car takes on the association of "assignedCar" with the multiplicity value of 0..1, so when the instance of RoadTrip exists, it can either have one instance of Car associated with it or no Cars associated with it.

INTERACTIONS

- In this case, a separate Caravan class with a multiplicity value of 0..* is needed to demonstrate that a RoadTrip could have multiple instances of Cars associated with it.
- Since one Car instance could have multiple "getRoadTrip" associations—in other words, one car could go on multiple road trips—the multiplicity value is set to 0..*

INTERACTIONS

- **Unidirectional association:** A slightly less common relationship between two classes.
- One class is aware of the other and interacts with it.
- Unidirectional association is modeled with a straight connecting line that points an open arrowhead from the knowing class to the known class.

SpeedEnforcer

+ courtLocation:str
+ date:date

+ calculateSpeed():return



Car

+ modelType:str
+ Doors:int
+ autoMaker:str

- Radio() <-- nobody touches my radio
+ windshieldWiper ()
+ changeDirection()

INTERACTIONS

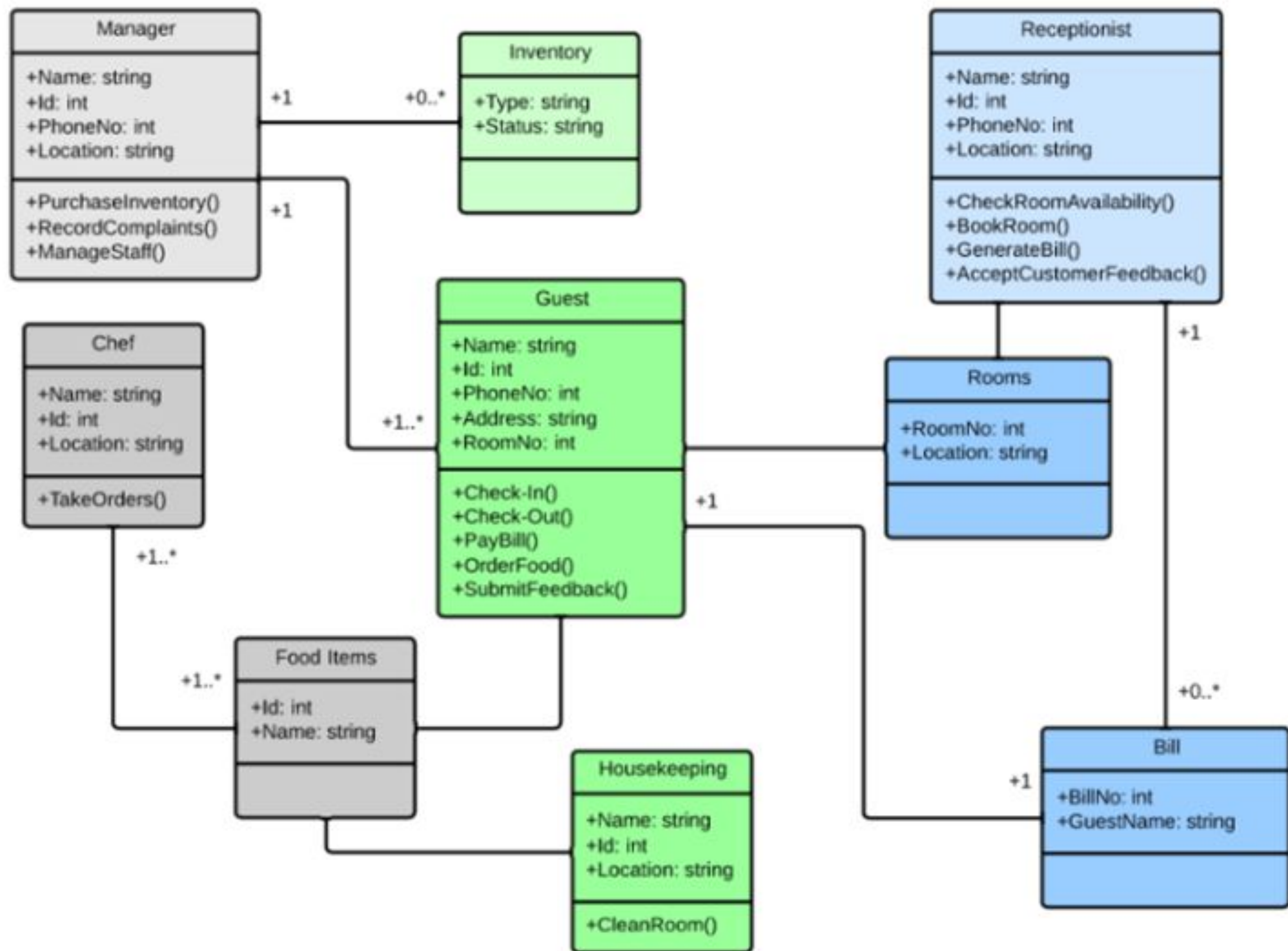
- As an example, on your road trip through Arizona, you might run across a speed trap where a speed cam records your driving activity, but you won't know about it until you get a notification in the mail. It isn't drawn in the image, but in this case, the multiplicity value would be $0..*$ depending on how many times you drive by the speed cam.

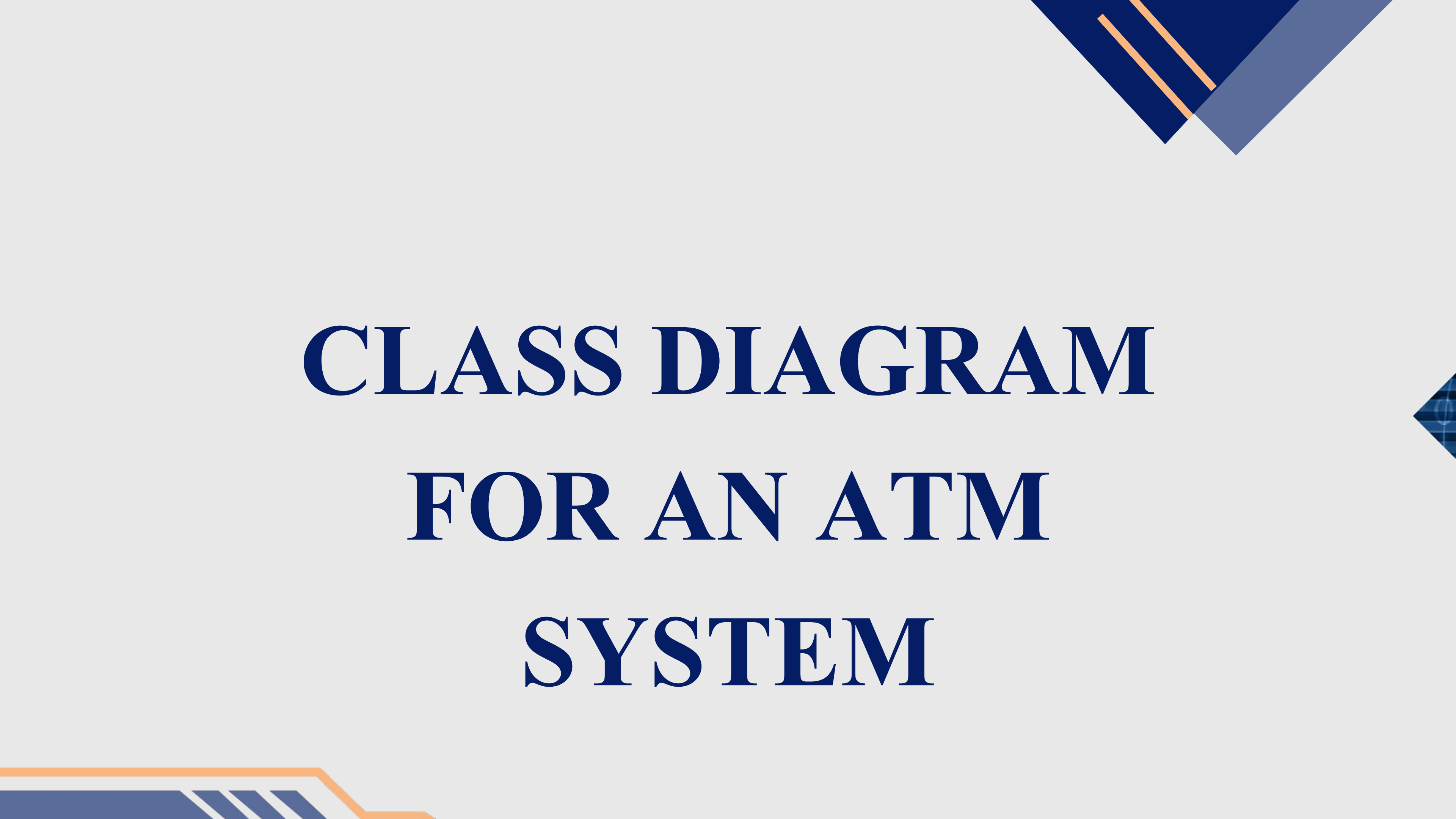
CLASS DIAGRAM EXAMPLES



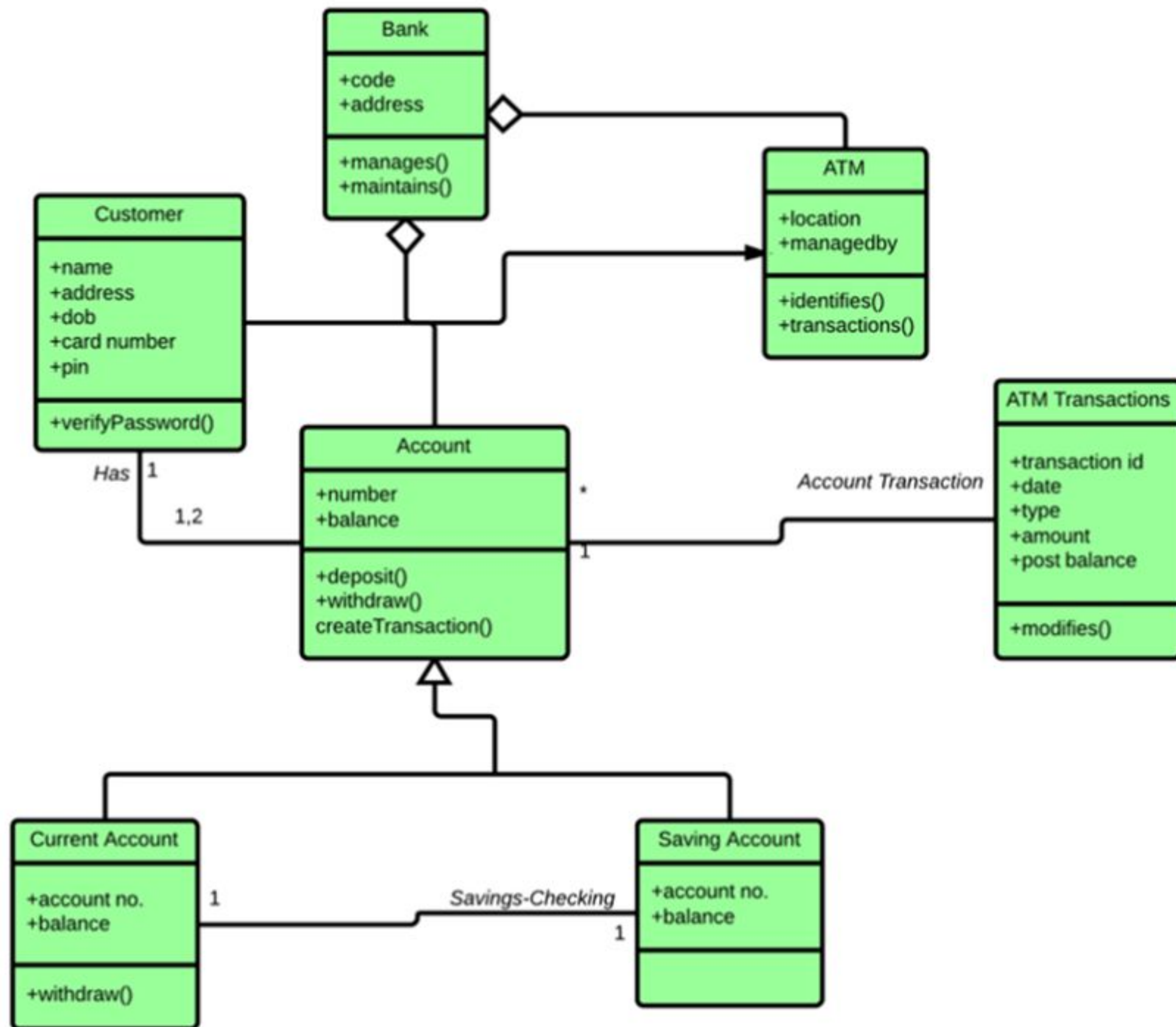


CLASS DIAGRAM FOR A HOTEL MANAGEMENT SYSTEM





CLASS DIAGRAM FOR AN ATM SYSTEM



THE END